

Dzień 1

Docker + Kubernetes podstawy

Maciej Krajewski · 2026

Plan dnia

1. **Docker**: image, optymalizacja Dockerfile, hardening, skanowanie
2. **K8s podstawy**: API, Pod, Service, Probes, Deployment, Namespace
3. **kubectl**: pierwsze komendy, context, debug
4. **Multi-container Pod**: sidecar pattern

→ Repo: `day1/`

Sekcja 1.1 — Dlaczego Docker?

- Konteneryzacja = uniformność środowiska
- "Działa u mnie" → "działa wszędzie"
- Odseparowanie aplikacji od OS
- Lekkie (vs VM): startup ms, overhead minimalny

Optymalizacja Dockerfile (D1/03)

Naiwny	Zoptymalizowany
FROM node:20 (1.2 GB)	FROM node:20-alpine AS build + distroless final
COPY . przed npm install	COPY package*.json first → cache layer
apt-get install vim git ...	tylko runtime deps
1 stage	multi-stage

Efekt: 1200 MB → 150 MB, build time -60%

→ day1/03_dockerfile_optimization/

Skanowanie obrazów: Trivy (D1/04)

```
trivy image --severity HIGH,CRITICAL --exit-code 1 myapp:v1
```

- OS packages CVE (apt/apk/yum)
- Language deps (npm, pip, go.mod)
- Misconfig (Dockerfile, K8s YAML)
- Secrets w obrazie

W CI: aquasecurity/trivy-action z exit-code → blokuje merge

→ day1/04_image_scanning_trivy/

Podpisywanie obrazów: Cosign + SBOM (D1/05)

```
cosign sign --key cosign.key myapp:v1  
cosign attest --predicate sbom.json --type cyclonedx myapp:v1  
cosign verify --key cosign.pub myapp:v1
```

- **Keyless** (OIDC) lepsze dla CI — krótko żyjące tożsamości
- **SBOM** = Software Bill of Materials (lista wszystkich komponentów)
- **Rekor** = transparency log (publiczny audit trail)

→ `day1/05_image_signing_cosign/`

Hardening obrazu (D1/06)

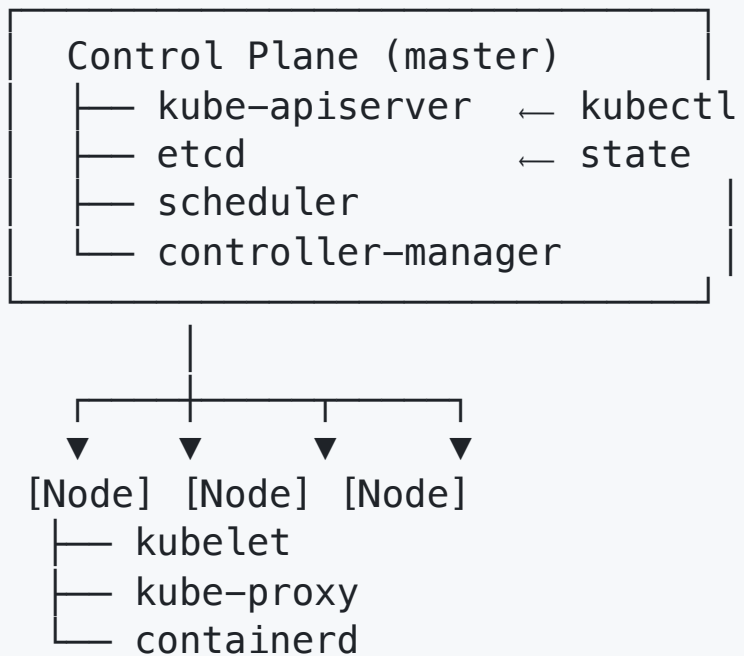
Checklist:

-  Distroless lub Alpine base
-  `USER nonroot`
-  Pinowane wersje (`nginx:1.27.0-alpine`)
-  Multi-stage build
-  `.dockerignore`
-  `HEALTHCHECK`
-  Hadolint w CI
-  Trivy + Cosign

Obraz musi być bezpieczny *nawet jeśli* uruchomi się bez SecurityContext klastra

→ `day1/06_hardening/`

Sekcja 1.2 — Architektura K8s



→ `day1/04_k8s/`

Pod = najmniejsza jednostka

- 1 lub więcej kontenerów
- Współdzielona sieć (localhost)
- Współdzielone wolumeny
- **Efemeryczny** — restart = nowy IP

```
kubectl apply -f pod.yaml  
kubectl get pods  
kubectl exec -ti myapp -- sh  
kubectl logs -f myapp  
kubectl port-forward pod/myapp 8080:80
```

→ day1/05_pod/

Multi-container Pod: sidecar (D1/13)

Wzorce:

- **Sidecar** — log shipper, metrics scraper
- **Ambassador** — local proxy do external service
- **Adapter** — format normalizer
- **Init container** — pre-start setup (D3/01)

Native sidecar (K8s 1.29+): `initContainers + restartPolicy: Always`

→ `day1/13_multi_container_pod/`

Probes: Liveness, Readiness, Startup (D1/09)

Probe	Fail action	Kiedy
Liveness	restart kontenera	deadlock
Readiness	usuń ze Service Endpoints	warming up
Startup	tłumi inne probe	wolne JVM apps

Antywzorzec: Liveness == Readiness — może spowodować boot loop

→ `day1/09_probes/`

Deployment + rolling update (D1/11)

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 0
```

- ReplicaSet pod spodem (history dla rollback)
- `kubectl rollout status / history / undo`
- Zmiana template → nowy ReplicaSet, stary → 0

Strategie wdrożeń (porównanie 6): D3/05 strategies.md

→ `day1/11_deployment/`

kubectl context (D1/14)

```
kubectl config get-contexts
kubectl config current-context
kubectl config use-context k3d-training
kubectl config set-context --current --namespace=myapp

# Bezpiecznie: explicit context per komenda destruktywna
kubectl --context=production delete pod ...
```

Slajd narzędzia (na końcu D5): kubectlx, kubens, k9s, kube-ps1

→ `day1/14_kubectl_context/`

Podsumowanie D1

- ✓ Docker: optymalizacja + hardening + Trivy + Cosign
- ✓ K8s: API, Pod, Service, Deployment, Namespace
- ✓ kubectl basics + context management
- ✓ Multi-container patterns

Domowa praca:

- Zbuduj swój obraz (D1/03 + 06)
- Skanuj Trivy (D1/04)
- Wdroż na lokalnym K3d (`./setup-cluster.sh`)

Jutro: Workloady, ConfigMaps, Secrets, AuthN/AuthZ, Gateway API

Dzień 2

Workloady, sieci, AuthN/AuthZ

Plan dnia

1. **Jobs / CronJobs**
2. **Storage:** Volumes, PV, PVC, StorageClass
3. **ConfigMaps + Secrets** (3 typy)
4. **AuthN/AuthZ:** Token, Cert, OIDC + RBAC
5. **Eksponowanie usług:** Gateway API + cert-manager
6. **StatefulSet, DaemonSet**

→ Repo: `day2/`

Jobs i CronJobs (D2/01-02)

Job = jednorazowe zadanie (DB migration, batch processing)

CronJob = harmonogram cron → tworzy Job

```
spec:
  parallelism: 5
  completions: 100
  backoffLimit: 6
```

Pułapki CronJob:

- `concurrencyPolicy: Forbid` — gdy poprzedni jeszcze trwa
- Joby kumulują się przy node down (`startingDeadlineSeconds`)

→ `day2/01_jobs/` , `day2/02_cronjob/`

Storage warstwy (D2/03)

	Co	Kiedy
<code>emptyDir</code>	temp per Pod	cache, scratch
<code>hostPath</code>	katalog z node	NIE produkcja
PV	klaster-level resource	admin pre-provisioning
PVC	user request	użycie z Pod
StorageClass	template dynamic provisioning	cloud (EBS, GCE disk, ...)

→ `day2/03_volume/`

ConfigMap (D2/05)

Dwa sposoby konsumpcji w Pod:

1. Env vars — **statyczne** (zmiana CM → restart Pod)
2. Mount jako pliki — **auto-refresh** ~60s

```
kubectl create configmap conf --from-file=./  
kubectl create configmap conf --from-env-file=env  
kubectl create configmap conf --from-literal=KEY=val
```

[stakater/Reloader](#) — auto-restart Deployment przy zmianie CM/Secret

→ `day2/05_configmap/`

Secrets — 3 typy (D2/10)

```
kubectl create secret generic      # Opaque
kubectl create secret docker-registry # imagePullSecrets
kubectl create secret tls          # cert + klucz dla Gateway/Ingress
```

⚠ Default = base64, NIE szyfrowane

Produkcja:

- etcd encryption at rest
- HashiCorp Vault (D4/04)
- SOPS (Git-encrypted)
- Sealed Secrets

→ day2/10_secrets/

AuthN/AuthZ flow

Request → AuthN → AuthZ → Admission → etcd

AuthN: kim jesteś?

- ServiceAccount Token (in-cluster)
- Client Cert (x509)
- OIDC (SSO)
- Webhook (custom)

AuthZ: co możesz?

- **RBAC** (najpopularniejsze)
- ABAC (legacy)
- Webhook

→ `day2/06_auth/`

RBAC w 30 sekund

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role                                # lub ClusterRole (cross-namespace)
metadata: { name: pod-reader, namespace: prod }
rules:
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list", "watch"]
---
kind: RoleBinding
subjects:
  - kind: User                            # lub ServiceAccount, Group
    name: alice
roleRef:
  kind: Role
  name: pod-reader
```

```
kubectl auth can-i list pods --as=alice
```

Strategie AuthN — kiedy?

Strategia	Kiedy	Lifetime
ServiceAccount BoundToken	in-cluster Pod	1h auto-refresh
Client cert (x509)	admin emergency	per cert (1y)
OIDC	human users (SSO)	1h refresh
Webhook	custom	depends

→ `day2/06_auth/01_token`, `02_cert`, `03_oidc`

Gateway API + cert-manager (D2/07)

Modern alternative dla Ingress (GA od K8s 1.29):

```
GatewayClass    ← infra admin
└─ Gateway      ← cluster operator
    └─ HTTPRoute ← app dev
```

- **cert-manager** dla automatycznych Let's Encrypt certs

→ `day2/07_gateway_api/` (Envoy Gateway implementation)

Gateway API — przykład

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata: { name: my-app }
spec:
  parentRefs: [{ name: training-gateway }]
  hostnames: [app.example.com]
  rules:
    - matches: [{ path: { type: PathPrefix, value: / } }]
      backendRefs: [{ name: my-app, port: 80 }]
```

Filter native: URLRewrite, RequestHeaderModifier, RequestMirror, RequestRedirect — bez annotations hell

StatefulSet (D2/08)

	Deployment	StatefulSet
Pod names	random	app-0 , app-1 , app-2
Ordering	równoległe	sekwencyjne
DNS	tylko Service	pod-N.service.ns.svc
Storage	shared	per-Pod PVC (volumeClaimTemplates)
Scale down	random	najwyższy ordinal

Use cases: PostgreSQL, MongoDB, Cassandra, Kafka, ElasticSearch

🤔 **Bazy w K8s — tak czy nie?** (link Google blog w README)

→ day2/08_statefulsets/

DaemonSet (D2/09)

Jeden Pod per node (per matching label).

Use cases:

- Log collector (Fluent Bit)
- Metrics agent (node-exporter, DCGM)
- CNI (Calico, Cilium)
- Security (Falco — D4/08)
- Storage agent (Longhorn)

```
nodeSelector:  
  nvidia.com/gpu.present: "true"    # tylko GPU nody (D5/07)
```

→ `day2/09_daemonsets/`

Podsumowanie D2

- ✓ Workloady: Jobs, CronJobs, StatefulSet, DaemonSet
- ✓ Storage stack: emptyDir → PV → StorageClass
- ✓ ConfigMaps + Secrets (3 typy)
- ✓ AuthN/AuthZ + RBAC
- ✓ Gateway API + cert-manager + Let's Encrypt

Jutro: Scheduling, autoscaling, NetworkPolicy, deployment strategies

Dzień 3

Scheduling, autoscaling, deployment

Plan dnia

1. **Init containers** — pre-start setup
2. **QoS, ResourceQuota, LimitRange**
3. **Metrics Server + HPA**
4. **Scheduling**: affinity, taints, topology spread
5. **Pod Priority + Preemption**
6. **Network Policies**
7. **Node maintenance** (drain, PDB)
8. **Strategie wdrożeń** (Canary deep dive)

→ Repo: `day3/`

Init Containers (D3/01)

Uruchamiają się **sekwencyjnie przed** main containers.

Use cases:

- Czekaj na Service / DB
- Schema migration
- Fetch secrets / config
- chown / chmod volumes

Native sidecars (K8s 1.29+): `initContainers + restartPolicy: Always` = sidecar startujący przed main

→ `day3/01_init_containers/`

QoS Classes (D3/02)

Klasa	Warunek	OOM-kill priority
Guaranteed	<code>requests == limits</code>	killed last
Burstable	<code>requests < limits</code>	medium
BestEffort	brak	killed first

Produkcja:

- Guaranteed: critical (DB, payment)
- Burstable: typowy web
- BestEffort: batch / dev

→ `day3/02_QoS/01_pod_limits/`

ResourceQuota + LimitRange

```
apiVersion: v1
kind: ResourceQuota
spec:
  hard:
    requests.cpu: "10"
    requests.memory: 20Gi
    nvidia.com/gpu: "4"
    pods: "50"
---
kind: LimitRange
spec:
  limits:
    - type: Container
      default: { cpu: 200m, memory: 256Mi }
      defaultRequest: { cpu: 100m, memory: 128Mi }
```

→ day3/02_QoS/02_limitrange , 03_resource_quota

Metrics Server (D3/03)

Wymóg dla `kubectl top`, HPA, VPA.

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/...  
# K3d/Kind: dodaj --kubelet-insecure-tls
```

```
kubectl top nodes  
kubectl top pods
```

⚠ Point-in-time, brak historii — Prometheus do długoterminowej (D5/04)

→ `day3/03_metrics_server/`

HPA — Horizontal Pod Autoscaler (D3/04)

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  scaleTargetRef: { name: my-app }
  minReplicas: 2
  maxReplicas: 50
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: { type: Utilization, averageUtilization: 70 }
```

- **Custom metrics** przez Prometheus Adapter
- **VPA** = vertical (zmienia `requests/limits`)
- **Cluster Autoscaler / Karpenter** = dodaje nody (D5/07)

→ `day3/04_HPA/`

Scheduling — 3 mechanizmy (D3/06)

Mechanizm	Cel
Node/Pod Affinity	preferuj/wymagaj lokalizacji
Taints + Tolerations	"nie planuj tu bez zgody"
Topology Spread	rozrzuć po domenach

Komplementarne, nie alternatywne.

Produkcja: GPU pool (`taint` + `nodeSelector`), spot instances (`taint`), multi-zone HA (TSC)

→ `day3/06_scheduling_rules/`

PriorityClass + Preemption (D3/07)

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata: { name: high-priority }
value: 1000
preemptionPolicy: PreemptLowerPriority
```

- Wyższy priority **wywłaszcza** niższe gdy klaster pełny
 - `preemptionPolicy: Never` — wysoki priority bez preemption
 - Built-in: `system-cluster-critical`, `system-node-critical`
- `day3/07_pod_priority/`

NetworkPolicy (D3/08)

Default: wszystkie Pody mogą rozmawiać z wszystkimi.

Default-deny + selective allow:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
spec:
  podSelector: { matchLabels: { app: backend } }
  policyTypes: [Ingress, Egress]
  ingress:
    - from:
      - podSelector: { matchLabels: { app: frontend } }
```

⚠ **Wymaga CNI z policy support:** Calico, Cilium (NIE Flannel)

→ day3/08_network_policy/

Node maintenance (D3/09)

```
kubectl cordon $NODE      # nie planuj nowych
kubectl drain $NODE \
  --ignore-daemonsets \
  --delete-emptydir-data
# patch / reboot
kubectl uncordon $NODE
```

PodDisruptionBudget — drain respektuje:

```
spec:
  minAvailable: 75%      # lub maxUnavailable: 1
  selector: { matchLabels: { app: critical } }
```

→ day3/09_node_maintenance/

Strategie wdrożeń — 6 wariantów (D3/05)

	Downtime	Resource	Rollback
Recreate	TAK	1×	trudny
Rolling	NIE	1.25×	wolny
Blue/Green	NIE	2×	natychmiastowy
Canary	NIE	1.x×	natychmiastowy
A/B	NIE	2×	per cohort
Shadow	NIE	2×	n/a (read-only)

Argo Rollouts = automatyzacja Canary z metryk Prometheus

→ `day3/05_Canary/strategies.md`

Canary — przykład Argo Rollouts

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
spec:
  strategy:
    canary:
      steps:
        - setWeight: 20
        - pause: { duration: 5m }
        - analysis: # auto-rollback jeśli error rate > 1%
            templates: [{ templateName: success-rate }]
        - setWeight: 50
        - pause: { duration: 10m }
        - setWeight: 100
```

→ `day3/05_Canary/canary-demo/`

Podsumowanie D3

- ✓ Init containers + QoS + ResourceQuota
- ✓ Metrics Server + HPA
- ✓ Affinity, taints, TSC
- ✓ PriorityClass + preemption
- ✓ NetworkPolicy + drain z PDB
- ✓ Strategie wdrożeń (porównanie 6)

Jutro: Security deep dive — PSA, RBAC zaawansowany, Vault, kube-bench, Trivy, Falco, OPA

Dzień 4

Security — pełen stack

Plan dnia

Pod-level hardening:

1. **Debug Pod** — kubectl debug, ephemeral containers
2. **Pod Security Admission + SecurityContext**
3. **Admission Controllers** + ValidatingAdmissionPolicy (CEL)
4. **HashiCorp Vault**

Cluster security tools:

5. **kube-bench** (CIS audit)
6. **Trivy Operator** (in-cluster scanning)
7. **Falco** (runtime detection)
8. **OPA / Gatekeeper** (policy as code)

Teoria: Service Mesh · Pentesty · Supply chain

→ Repo: `day4/`

Debug Pod (D4/01)

Distroless image = brak shella. Co teraz?

```
# Ephemeral container (in-place)
kubectl debug -it my-pod --image=nicolaka/netshoot \
  -- tcpdump -n port 8080

# Copy-to (klon z innym image)
kubectl debug -it my-pod --image=ubuntu \
  --share-processes --copy-to=my-pod-debug
```

→ day4/01_debug_pod/

Pod Security Admission (D4/02)

3 standardy × 3 modes per namespace:

Standard	Co
privileged	wszystko (default)
baseline	minimal restrictions (no host network/PID)
restricted	hardened (non-root, drop capabilities, seccomp)

```
apiVersion: v1
kind: Namespace
metadata:
  labels:
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/audit: restricted
    pod-security.kubernetes.io/warn: restricted
```

→ day4/02_psa_security/

Admission Controllers (D4/03)

Request → AuthN → AuthZ → Mutating Admission → Validating Admission → etcd

Trzy podejścia:

1. **Built-in** (PSA, ResourceQuota, LimitRanger, ...)
2. **Webhooks** (Vault, Gatekeeper, Kyverno)
3. **ValidatingAdmissionPolicy + CEL** (K8s 1.30+ stable, in-cluster, lekkie)

```
validations:  
  - expression: "object.spec.replicas >= 2"  
    message: "Min 2 repliki"
```

→ `day4/03_Admission_Controllers/`

HashiCorp Vault (D4/04)

Production secret management — alternatywa dla K8s Secrets.

3 wzorce integracji:

1. **CSI Driver** — secrets mounted jako pliki (`csisecret.yaml`)
2. **CSI + env** — secrets jako env vars
3. **Vault Agent Injector** — sidecar refreshujący secrets

```
helm install vault hashicorp/vault --version 0.24.1
```

→ `day4/04_vault/`

SecurityContext (D4/05)

```
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
    seccompProfile: { type: RuntimeDefault }
  containers:
    - securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities: { drop: ["ALL"] }
```

Defense in depth: image hardening + SecurityContext + PSA + OPA

→ `day4/05_security_context/`

kube-bench — CIS audit (D4/06)

```
kubectl apply -f kube-bench-job.yaml
kubectl logs job/kube-bench
# [PASS] 1.1.1 ...
# [FAIL] 5.1.5 default ServiceAccounts not actively used
# [WARN] 5.1.6 ...
```

CIS Kubernetes Benchmark = ~150 best practices security

- Sekcja 1-3: Control Plane
- Sekcja 4: Worker
- Sekcja 5: Policies (RBAC, PSA, NetworkPolicy)

→ `day4/06_kube_bench/`

Trivy Operator (D4/07)

Ciągłe skanowanie obrazów uruchomionych Pod-ów.

```
helm install trivy-operator aqua/trivy-operator -n trivy-system  
kubectl get vulnerabilityreports -A  
kubectl get configauditreports -A  
kubectl get rbacassessmentreports -A  
kubectl get exposedsecretreports -A
```

Grafana dashboard 17813 — Trivy reports visualization

→ `day4/07_trivy_k8s/`

Falco — runtime security (D4/08)

eBPF-based runtime detection. Wykrywa:

- Shell w produkcji
- Zapis do `/etc/`
- Mount sensitive paths
- Nieoczekiwany egress
- ECC errors / hardware events

```
- rule: Custom - Write to /etc
  condition: evt.is_open_write and fd.name startswith /etc/ and container
  output: "Zapis do /etc (user=%user.name file=%fd.name container=%container.name)"
  priority: WARNING
```

Falcosidekick → Slack / PagerDuty / Elastic / Kafka

→ `day4/08_falco/`

OPA / Gatekeeper (D4/09) — ConstraintTemplate

Policy as code (Rego). Najpierw definiujemy klasę policy:

```
kind: ConstraintTemplate
spec:
  crd: { spec: { names: { kind: K8sRequiredLabels } } }
  targets:
    - rego: |
        violation[{"msg": msg}] {
          missing := required - provided
          count(missing) > 0
          msg := sprintf("missing labels: %v", [missing])
        }
```

ConstraintTemplate generuje **nowy CRD** (`K8sRequiredLabels`) — meta-CRD pattern.

OPA / Gatekeeper — Constraint (instancja)

Z ConstraintTemplate wygenerowanego CRD tworzymy konkretną politykę:

```
kind: K8sRequiredLabels
spec:
  match: { kinds: [{ apiGroups: [""], kinds: [Pod] }] }
  parameters: { labels: ["owner"] }
```

Każdy Pod bez labela `owner` → admission webhook deny.

Alternatywa: Kyverno (YAML zamiast Rego, prostsza dla większości use cases)

→ `day4/09_opa_gatekeeper/`

Service Mesh — kiedy? (D4/10)

⚠ Tylko teoria, nie ćwiczenie

	Istio	Linkerd	Cilium SM
Sidecar	Envoy	linkerd2-proxy (Rust)	eBPF + Envoy L7
Złożoność	wysoka	niska	średnia
Latency overhead	5-10ms	1-3ms	minimal

Kiedy: 50+ microservices, zero-trust, mTLS compliance

NIE: monolit + 2-3 services, brak platform team

→ `day4/10_service_mesh_intro.md`

Pentesty K8s (D4/11)

⚠ Tylko teoria + linki

Faza 1: Recon (kube-hunter)

Faza 2: Compromised Pod → SA token, Kubelet API

Faza 3: Lateral movement (brak NetworkPolicy)

Faza 4: Persistence (mutating webhook, hidden CronJob)

Narzędzia: kube-hunter, peirates, amicontained, kubescape

Frameworks: MITRE ATT&CK Containers, NSA/CISA K8s Hardening Guide

→ `day4/11_cluster_pentest_intro.md`

Supply Chain (D4/12)

Cross-link D1 (Cosign + SBOM + Trivy) → D4 (admission policy):

```
git → CI build → Trivy scan → Cosign sign + SBOM → registry
                        ↓
                Sigstore Policy Controller
                        ↓
                K8s admission accept/deny
                        ↓
                Trivy Operator runtime scan
```

SLSA framework: L1-L4 dojrzałości supply chain

→ `day4/12_supply_chain.md`

Podsumowanie D4

- ✓ PSA + SecurityContext (Pod-level hardening)
- ✓ Vault (production secrets)
- ✓ kube-bench (compliance audit)
- ✓ Trivy Operator (runtime CVE scan)
- ✓ Falco (anomaly detection)
- ✓ OPA/Gatekeeper (policy enforcement)
- ✓ Service Mesh, Pentesty, Supply Chain (theory)

Jutro: Helm, monitoring, kubeadm, **AI/GPU na K8s**

Dzień 5

Helm, monitoring, kubeadm, AI/GPU

Plan dnia

1. **Helm** — package manager + custom charts
2. **Monitoring** — Prometheus + Grafana + Loki
3. **Kubeadm** — klaster od zera (HA: 3 CP + 3 worker)
4. **AI/GPU na K8s** — fake-gpu-operator, MIG, Kueue, DCGM, production practices
5. **Narzędzia developerskie** (cheatsheet)

→ Repo: `day5/`

Helm (D5/02)

Chart = paczka + templates + values

Release = instancja chartu w klastrze

```
helm install my-app ./mychart
helm upgrade --install --atomic my-app ./mychart
helm rollback my-app 1
helm history my-app
helm template ./mychart --debug # render bez install
```

Subkatalogi:

- 01_install_chart/ — instalacja z workshop repo
- 02_happy_panda/ — chart z artifacthub.io (OCI)
- 03_own_chart/ — własny chart (Python+Redis)

→ day5/02_helm/

Helm chart structure

```
mychart/
├── Chart.yaml           # metadata
├── values.yaml          # defaults
├── templates/
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── ingress.yaml
│   ├── _helpers.tpl     # reusable snippets
│   └── NOTES.txt        # post-install message
└── charts/              # subcharts
```

--atomic w CI = automatic rollback przy fail

Prometheus stack (D5/04)

```
helm install kube-prometheus-stack prometheus-community/...
```

Zawiera:

- **Prometheus Operator** (CRD: ServiceMonitor, PrometheusRule)
- **Prometheus** (TSDB + scraping)
- **AlertManager** (routing alertów)
- **Grafana** (preinstall'd dashboards)
- **node-exporter** (DaemonSet, per-node metrics)
- **kube-state-metrics** (K8s objects metrics)

→ `day5/04_monitoring_alerting/`

Loki — central logging

```
helm install loki grafana/loki-stack
```

Like Prometheus but for **logs** — LogQL podobny do PromQL.

```
{namespace="default"} |= "error"  
{pod=~"nginx-.*"} | json | response_time > 1000
```

Lekki, kompatybilny z Grafana, **chunks compression** = tani storage.

Alternatywy: ELK (Elasticsearch+Logstash+Kibana), OpenSearch, Splunk, Datadog

Custom Prometheus alerts

```
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
spec:
  groups:
    - name: app.alerts
      rules:
        - alert: HighErrorRate
          expr: |
            sum(rate(http_requests_total{status=~"5.."}[5m])) > 0.05
          for: 5m
          annotations:
            summary: "Error rate > 5% przez 5 min"
            runbook_url: https://runbooks.example.com/high-error
```

AlertManager → Slack / PagerDuty / Opsgenie / email / webhook

kubeadm: klaster od zera (D5/06)

Cel: 3 CP + 3 worker + kube-vip + Cilium

```
# Na każdym node:  
sudo ./prepare.sh  
  
# Pierwszy CP:  
sudo kubeadm init --config kubeadm-config.yaml --upload-certs  
cilium install --version 1.17.6  
  
# Reszta CP:  
sudo kubeadm join ... --control-plane --certificate-key ...  
  
# Workery:  
sudo kubeadm join ...
```

Alternatywy: kops (AWS), kubespray (Ansible), k3s/RKE2 (lekki), EKS/GKE/AKS (managed)

→ `day5/06_install/`

Sekcja 10 — AI / GPU na K8s

🔥 Najszybciej rosnący segment K8s w 2024-2026.

Wyzwanie szkolenia: nie wymagamy fizycznego GPU.

Rozwiązanie: [fake-gpu-operator](#) (Run:AI) — emuluje cały stos NVIDIA na CPU-only K3d/Kind klastrze.

✅ Scheduling logic, MIG, multi-GPU, taints, autoscaling

❌ Faktyczne CUDA workloady (mock metrics)

→ `day5/07_ai_gpu/`

GPU stack na K8s

```
[Pod: nvidia.com/gpu: 1]
  ↓
[K8s scheduler] ← respektuje extended resources
  ↓
[NVIDIA Device Plugin] ← DaemonSet, gRPC do kubelet
  ↓
[NVIDIA GPU Operator] ← orkiestracja całego stacka:
  ├── Driver DaemonSet
  ├── Device Plugin
  ├── Container Toolkit (mounts CUDA libs)
  ├── DCGM Exporter (metrics → Prometheus)
  ├── MIG Manager
  └── GPU Feature Discovery
```

→ `day5/07_ai_gpu/01_install_fake_gpu/`

MIG — Multi-Instance GPU

A100 (80GB) podzielony hardware-level na "mini-GPU":

Profile	SM	Memory	Max instances
1g.10gb	1/7	10 GB	7
3g.20gb	3/7	20 GB	2
7g.80gb	7/7	80 GB	1 (cały)

```
resources:  
  limits:  
    nvidia.com/mig-1g.5gb: 1
```

Strategia: `single` (cały klaster jeden profile) vs `mixed` (różne)

→ `day5/07_ai_gpu/04_mig_demo/`

Kueue — job queueing dla AI/ML

⚠️ Tylko przegląd (zbyt obszerne na 1 dzień)

`ResourceFlavor` → `ClusterQueue` (quota) → `LocalQueue` → `Workload`

Use case: 3 teamy współdzielą 16 GPU. Team A nie używa? Team B borrowuje. Sprawiedliwe + fair sharing.

Native integracje: Job, JobSet, MPIJob, RayJob, PyTorchJob

Alternatywy: Volcano, YuniKorn

→ `day5/07_ai_gpu/05_kueue_intro.md`

DCGM Exporter + Prometheus

Najważniejsze metryki:

- `DCGM_FI_DEV_GPU_UTIL` — % SM utilization
- `DCGM_FI_DEV_FB_USED` — memory used
- `DCGM_FI_DEV_GPU_TEMP` — thermal (alert > 83°C)
- `DCGM_FI_PROF_PIPE_TENSOR_ACTIVE` — Tensor Cores activity
- `DCGM_FI_DEV_ECC_DBE_V0L_TOTAL` — ECC errors (alert > 0)

Grafana dashboard 12239 — DCGM Exporter Dashboard

→ `day5/07_ai_gpu/06_dcgm_concepts.md`

GPU production practices

- ✓ **Dedicated node pool** (taint `nvidia.com/gpu=true:NoSchedule`)
- ✓ **Spot instances** (70-90% taniej, jeśli checkpointing)
- ✓ **Karpenter** zamiast Cluster Autoscaler (szybszy, multi-instance type, spot fallback)
- ✓ **Image pre-loading** (Spegel) — 5GB pull × 100 nodów = 500GB
- ✓ **Cost monitoring** (OpenCost / Kubecost)

→ `day5/07_ai_gpu/07_production_practices.md`

Sekcja 11 — Narzędzia developerskie (CLI)

(Bez ćwiczeń w repo, tylko cheatsheet)

Tool	Co robi
kubectx	szybsze switch context
kubens	szybsze switch namespace
k9s	terminal UI dla K8s
kube-ps1	shell prompt z context+namespace
stern	multi-pod log tail

Sekcja 11 — Narzędzia developerskie (GUI / dev)

Tool	Co robi
Lens	desktop GUI (multi-cluster)
Headlamp	open-source alternatywa Lens
Telepresence	local dev z proxy do remote cluster
VS Code K8s extension	manifesty, snippets, debug Pod

Co dalej? (continuous learning)

- **Certyfikacje:** CKA, CKAD, CKS (Linux Foundation)
- **CNCF Landscape** — `landscape.cncf.io`
- **TGI Kubernetes** (live show on YouTube)
- **KubeCon** talks (recordings free na YouTube)
- **The New Stack, Daily.dev** dla bieżących newsów

Podsumowanie 5-dniowego kursu

Dzień	Tematy główne
D1	Docker hardening + K8s podstawy
D2	Workloady, AuthN/AuthZ, Gateway API
D3	Scheduling, autoscaling, deployment
D4	Security pełen stack (PSA, Vault, OPA, Falco, Trivy)
D5	Helm, monitoring, kubeadm, AI/GPU

Repo: github.com/<TWOJ_LOGIN>/k8s-training-2026

Dziękuję! 🎉